Meta-HCapsNet: A Meta-Learning Enhanced Hypergraph Capsule Network for Accurate and Interpretable Stock Price Forecasting

Yucheng Wang[†a], Zhunyan He[†b] and Kun Yu[c,*]

[a]School of Computing, National University of Singapore, 119077, Singapore
[b]School of Finance, Wenzhou Kean University , Wenzhou, 066004, China
[c]Baidu (China) Co., Ltd, Haidian District, 100085, China

ARTICLE INFO

Keywords:
Stock price forecasting
Capsule networks
Hypergraph neural networks
Meta-learning
Interpretability

ABSTRACT

Stock price sequence prediction is highly challenging due to the high noise, non-stationarity, and complex high-order interdependencies in financial time series. The existing deep learning models encounter difficulties in clearly capturing the group-level synergy effects beyond pairwise relationships, and also lack adaptability to different market environments as well as the interpretability required for practical decision-making. This paper proposes Meta-HCapsNet, an attention-enhanced capsule network incorporating hypergraph structure for stock price forecasting. The model introduces three key innovations: a lightweight Meta-Adapter that dynamically generates hypergraph construction parameters and capsule routing priors based on input statistical features to enable rapid meta-learning adaptation; a novel Hypergraph Capsule Routing layer that embeds high-order group correlations into the iterative dynamic routing process of capsules; and an adaptive attention pooling mechanism that provides intuitive interpretability through activation patterns. Extensive experiments on three real-world Chinese stock datasets from BYD, CATL, and CSSC demonstrate that Meta-HCapsNet consistently outperforms seven state-of-the-art baseline models, including iTransformer and MEAformer, achieving up to 54% reduction in MAE as well as significant reductions in RMSE and MAPE across short- and long-term horizons. Ablation studies confirm the effectiveness of each component, few-shot adaptation experiments verify superior generalization with limited data, and interpretability analysis shows that capsule activation intensities strongly correspond to meaningful market phases such as rising and retracement, demonstrating the model's accuracy, adaptability, and transparency in modeling complex financial dynamics.

1. Introduction

Financial time series prediction has always been a research hotspot in the interdisciplinary field of artificial intelligence and financial engineering due to its core role in quantitative investment, risk hedging, and market supervision. However, the financial market, as a complex dynamical system, exhibits extremely high levels of noise, non-stationarity, and multi-factor coupling, making it extremely challenging to accurately capture the patterns of price fluctuations[1, 2]. Traditional statistical and econometric models and early machine learning methods often encounter limitations in extracting deep market dynamics features due to the bottleneck of manual feature engineering when dealing with massive high-dimensional and nonlinear financial data[3].

In recent years, the rapid development of deep learning technology has completely changed this situation. Long Short-Term Memory networks (LSTM) and their variants once dominated sequential modeling, but their serial computing characteristics limited their ability to capture long-term dependencies[4]. Subsequently, the Transformer architecture achieved breakthroughs in long sequence prediction tasks through the self-attention mechanism. For instance, PatchTST[5] proposed patching time series to retain local semantics, significantly improving prediction accuracy; iTransformer[6] solved the problem of feature fusion in multi-variable sequences by reversing dimensions and applying the attention mechanism on the variable dimension. Additionally, DeepONet[7]and various hybrid models[8] have also been proposed to further enhance the model's ability to fit nonlinear patterns.

Although Transformer-based models excel in capturing long-range temporal dependencies, they often overlook the complex spatial topological structure among financial entities. Stocks in the financial market do not fluctuate independently but form a closely interconnected network through supply chains, equity relationships, or macro policies. To address this issue, graph neural networks (GNNs) have been widely adopted, such as STGAT[9] and CIRGNN[10], which explicitly model the pairwise relationships between stocks using graph attention mechanisms. However, the interconnections in the real market often exhibit "group synergy" effects, such as collective movements within the same concept sector. This higher-order dependency beyond pairwise relationships cannot be effectively described by simple graph structures. Therefore, the hypergraph theory has gained attention. Recent studies, such as FactorGCL[11], indicate that hypergraph structures can more accurately capture the complex nonlinear group relationships among multiple nodes.

However, the existing models, while striving for high accuracy, often encounter two key drawbacks: one is their

*Corresponding author: Kun Yu (Email: yukun02@baidu.com)
**This authors contribute equally in this paper.(Yucheng Wang[†] and Zhunyan He[†].)
wangyucheng@u.nus.edu (Y. Wang[†]); zhunyanh@kean.edu (Z. He[†]); yukun02@baidu.com (K. Yu)
ORCID(s):

lack of adaptability, making it difficult to cope with the rapid changes in market styles. The second issue is the lack of interpretability. The "black box" nature of deep neural networks severely hinders their credibility in financial decision-making. To address the aforementioned challenges, this paper presents a meta-learning capsule network with a hypergraph structure(Meta-HCapsNet) for stock sequence prediction. This model incorporates three core innovations: Firstly, a lightweight Meta-Adapter is designed based on the idea of meta-learning. It dynamically generates hypergraph construction parameters and routing priors according to the statistical features of the input sequence, enabling the model to adapt to different market styles; Secondly, an innovative hypergraph Capsule Routing layer is proposed, which embeds the hypergraph structure deeply into the dynamic routing process of capsules. In the iterative update, it explicitly encodes the high-order group relationships between time points, compensating for the shortcomings of traditional Transformers and GNNs in modeling high-order correlations; Finally, an adaptive attention pooling mechanism is implemented for decoding from high-level capsules to global predictions, and the activation patterns of capsules provide intuitive explanations of market states.

2. Literature review

Stock market prediction, as an interdisciplinary field combining financial engineering and computational intelligence, is characterized by its complexity due to the high noise in market microstructure, the nonlinear transmission mechanism of macroeconomic factors, and the group irrationality of investor behavior[12, 13]. In recent years, with the continuous evolution of deep learning architectures, the mature application of graph neural network theory, and the breakthrough development of generative artificial intelligence technology, stock prediction research has shifted from traditional single-variable time series modeling to the construction of a systematic framework for multimodal data fusion, dynamic relationship graph modeling, and interpretable intelligent decision-making.

2.1. Traditional machine learning methods

The traditional machine learning research for stock prediction mainly relied on shallow models such as Support Vector Machines (SVM)[14] and Random Forests (RF)[15, 16], and focused on feature optimization and integration strategies. For instance, Nti et al.[17] evaluated the comprehensive effectiveness of ensemble learning in stock prediction in the Journal of Big Data and found that the Stacking method was significantly superior to a single model. Alotaibi[18] proposed a hybrid feature selection method based on red deer-gray wolf optimization, achieving a 9.2% accuracy improvement in the Saudi stock market.

Although ensemble learning significantly enhances the robustness of predictions through model combination, its effectiveness is highly dependent on the quality of input features. Therefore, feature engineering, as another core pillar determining the upper limit of model performance, has achieved breakthrough progress in recent years with the empowerment of deep learning[19]. Long et al.[20] proposed a deep learning feature engineering framework that broke through the limitations of traditional technical indicators' linear combination. It automatically extracted high-order abstract features of the price sequence through the Restricted Boltzmann Machine (RBM). Paiva et al.[21] in Expert Systems with Applications restructured the feature engineering process from the perspective of portfolio optimization, combining the Markowitz mean-variance framework with machine learning prediction. Through the output yield distribution of the model, the optimal weight vector was directly generated. In an empirical study at the São Paulo Stock Exchange in Brazil, this integrated method increased the annualized return of the strategy.

2.2. Deep learning methods

While feature engineering lays a solid foundation of high-quality data for the model, the continuous innovation of the deep learning architecture itself has become another core engine driving breakthroughs in prediction performance. In recent years, it has achieved leapfrog development in three dimensions: long short-term memory network(LSTM) optimization, convolutional neural network (CNN) spatio-temporal modeling, and Transformer paradigm innovation.

The LSTM has further consolidated its dominant position in time series modeling due to its gating mechanism. Saurabh[22] was the first to apply LSTM to predict the daily stock prices of a specific company. Compared with traditional machine learning and artificial neural network models, it was verified that LSTM can accept and consider the sequence of input data, which is conducive to the persistence of the data. Md et al.[23] innovatively adopted a three-layer stacked architecture of sequential LSTM and introduced the sine cosine algorithm (SCA) to optimize the hyperparameters of the initial learning rate and batch size. In the daily price predictions of three technology stocks (Apple, Google, and Microsoft), it achieved a reduction of over 60% compared to traditional LSTM.

After the convolutional neural network (CNN) was transferred from computer vision to financial time series analysis, its local receptive field and parameter sharing characteristics have demonstrated unique advantages. The BiCuDNNLSTM-1dCNN hybrid model proposed by Kanwal et al.[24] represents the current technological frontier. This architecture uses bidirectional CUDA-accelerated LSTM to extract long-term dependencies, followed by 1D-CNN to capture short-term price patterns. On real datasets, it outperforms a single LSTM model by 9% and reduces training time by 40%. Prasad et al.[25] explored the dimension expansion of CNN and designed 2D-CNN to process price-volume heat maps, and 3D-CNN to construct a three-dimensional tensor of time-price-technical indicators. Through the three-dimensional representation of 82 technical features, the prediction accuracy was improved by 3-11% on Nasdaq 100 component stocks.

The self-attention mechanism of the Transformer model has completely transformed the traditional time-series modeling approach by demonstrating its superiority in handling long-range dependencies. Sun et al.[26] developed a dual-channel CNN-LSTM-Attention fusion model that employs a multi-head attention mechanism to dynamically allocate weights for different time steps. In the CSI300 index prediction, this model outperformed the traditional model by 15%. However, the most revolutionary development is the MEAformer proposed by Huang et al.[27] discard the traditional self-attention mechanism and introduces an external attention (External Attention) mechanism, projecting the time series to an external memory unit and reducing the computational complexity from $O(n^2)$ to $O(n)$, enabling sequence input of up to 2048 time steps. In long-term prediction tasks, the error is reduced by 15%, while memory usage is decreased by 50%.

2.3. Graph neural network methods

The inter-industry correlations, supply chain relationships, and investor co-investment patterns among stocks form a complex dynamic graph structure. Kim and Park [28] proposed STAD-GCN, which captures the evolution of the graph structure through a time attention mechanism. Each trading day builds a stock correlation graph, spatial graph convolution aggregates neighbor information, and time convolution captures sequence dynamics, achieving higher accuracy compared to static graph models. Yang et al. [29] constructed the MDF-DMC model, which introduces multi-view features and updates dynamic market correlations through gated recurrent units (GRUs). The prediction accuracy on cross-industry stock pools increased by 6.8%, demonstrating the effectiveness of heterogeneous information fusion. Xiang et al.[30] proposed the Time-Heterogeneous Heterogeneous Graph Neural Network (THGNN) treating stocks, industries, regions, and concepts as heterogeneous nodes. It captures high-order correlations of "stock - industry - stock" through meta-paths. In the prediction of the entire A-share market, this model reduces the prediction error for small and medium-sized stocks, proving its advantage in data-sparse scenarios. Huynh et al.[31] proposed the Multi-order Dynamics Integration model, which integrates first-order and second-order dynamics. It learns individual stock-specific patterns through an internal dynamic encoder (IDE) and provides new ideas for high-frequency trading.

3. Methodology

The overall architecture of Meta-HCapsNet is a hierarchical information processing flow, as shown in Figure 1. The model starts with raw multi-dimensional financial time series data, first extracting local features through a one-dimensional convolutional layer, and then converting them into primary capsule representations containing pose information. Next, a lightweight meta-adapter analyzes the statistical features of the primary capsule sequence, dynamically generating hypergraph construction parameters and capsule routing prior parameters. These parameters are then sent to the hypergraph capsule routing layer-the core innovative module of this model, which deeply integrates the hypergraph structure into the dynamic routing process of capsules, generating high-level capsules capable of representing complex high-order patterns through iterative optimization. Finally, the high-level capsules are aggregated into a global representation through an adaptive attention pooling mechanism and multi-step predictions are made through a multi-layer perceptron. The entire process achieves end-to-end learning from raw data to prediction results, while maintaining the structured and interpretable nature of the internal computational process.

3.1. Input representation

The input of the model is a standard financial time series matrix. Suppose we select the data of the past T trading days, and each trading day contains F features (such as opening price, highest price, lowest price, closing price, trading volume, etc.), then the input matrix is represented as $X \in R^{T\times F}$, where T is the step size of the historical time window and F is the dimension of the input features. To capture the patterns within a local time window, we first use a one-dimensional convolution layer to extract features from the input sequence:

$$U = \mathbf{ReLU}(W_c * X + b_c) \tag{1}$$

where $W_c \in R^{k\times F\times D}$ where $*$ represents the one-dimensional convolution operation; $W_c \in R^{k\times F\times D}$ is the convolution kernel weight, where k is the size of the convolution kernel and defines the width of the receptive field; $b_c \in R^D$ is the bias term, and D is the number of channels of the output feature map. The activation function **ReLU** is used to introduce the ability of nonlinear transformation. The output U can be regarded as a set of feature vectors along the time axis, rich in local information, which lays the foundation for the subsequent capsule representation.

3.2. Generation of primary capsules

In order to transform scalar features into vector representations that can simultaneously encode the existence probability of entities and their posture attributes, we introduce the basic units of the capsule network[32, 33]. First, the convolutional feature U is reshaped into a three-dimensional tensor $V^{(0)} \in R^{T\times C\times d_p}$, where $v_{t,c}^{(0)} \in R^{d_p}$ represents the c-th primary capsule vector at the t-th time step. To ensure that the magnitude of the capsule vector can represent the probability of the existence of the entity and retain the attitude information represented by its direction, we apply the squash activation function to each capsule:

$$\mathbf{Squash}(v^{(0)}) = \frac{\|v^{(0)}\|^2}{1+\|v^{(0)}\|^2} \cdot \frac{v^{(0)}}{\|v^{(0)}\|} \tag{2}$$

This function maintains the direction of the input vector u unchanged, but compresses its magnitude to the interval $[0, 1)$. The compressed magnitude can be intuitively interpreted as the confidence level of the existence of a certain

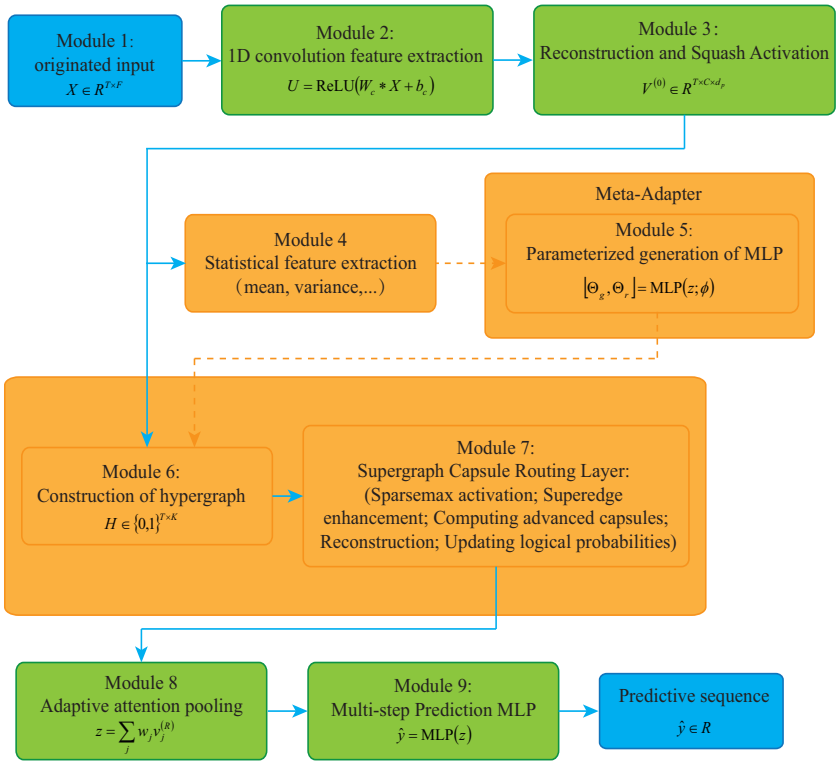

Figure 1: Overall Architecture of Meta-HCapsNet

local market pattern detected by this capsule, while the direction of the vector encodes the specific attributes of that pattern.

3.3. The construction of Meta-Adapter

In order for the model to be able to perceive different market conditions and dynamically adjust its internal parameters, we have designed a meta-adapter (Meta-Adapter). The meta-adapter first extracts a set of low-dimensional but informative statistical features $z \in R^{d_s}$ from the primary capsule sequence $V^{(0)}$, which include five parts: mean feature μ_d, variance feature σ_d^2, autocorrelation feature ρ_τ, trend intensity β and volatility vol:

$$\begin{cases} \mu_d = \frac{1}{T \cdot C} \sum_{t=1}^{T} \sum_{c=1}^{C} v_{t,c,d}^{(0)}, \quad d = 1, \cdots, d_p \\ \sigma_d^2 = \frac{1}{T \cdot C} \sum_{t=1}^{T} \sum_{c=1}^{C} (v_{t,c,d}^{(0)} - \mu_d)^2 \\ \rho_\tau = \frac{\sum_{t=\tau+1}^{T} (v_t - \bar{v})(v_{t-\tau} - \bar{v})}{\sum_{t=1}^{T} (v_t - \bar{v})^2}, \quad \tau = 1, 2 \\ \beta = \left| \frac{\sum_{t=1}^{T} (t - \bar{t})(v_t - \bar{v})}{\sum_{t=1}^{T} (t - \bar{t})^2} \right| \\ vol = \sqrt{\frac{1}{T-1} \sum_{t=1}^{T} (v_t - \bar{v})^2} \end{cases} \tag{3}$$

where v_t represents the average of all capsule vectors at time step t, that is $v_t = \frac{1}{C} \sum_{c=1}^{C} v_{t,c}^{(0)}$; $\bar{v}$ is the mean of all v_t values. In this paper, only the autocorrelation coefficients for lags 1 and 2 are calculated. $\bar{t} = \frac{T+1}{2}$ is the mean value of the time index.

By combining all these statistical quantities, we obtain the variable z, which summarizes the macroscopic characteristics such as the stationarity, trend, and volatility of the current input sequence. Subsequently, a multi-layer perceptron (MLP) with a parameter ϕ processes this summary vector and outputs two sets of key dynamic parameters:

$$[\Theta_g, \Theta_r] = \mathbf{MLP}(z; \phi) \tag{4}$$

where $\Theta_g = \{K, \tau, \lambda_f, \lambda_t, \tau_t\}$ is the parameter set for constructing the hypergraph. K specifies the number of hyperedges to be generated; τ is the threshold for the similarity between nodes, used to determine whether to connect; λ_f and λ_t respectively balance the weights of feature similarity and temporal proximity in the similarity calculation; τ_t controls the speed of time decay. These parameters jointly determine the granularity of the hypergraph structure. $\Theta_r \in R^{C \times M}$ is the routing prior parameter matrix, where $N = T \times C$ represents the total number of primary capsules and M is the number of advanced capsules to be generated. This matrix provides an initial propensity score for the connection between each pair of primary and advanced capsules, which will be used to initialize the routing log probability b_{ij}, thereby guiding the model to focus on specific pattern combinations. ϕ is the learnable parameter of the meta-adapter and is optimized through the meta-learning training strategy.

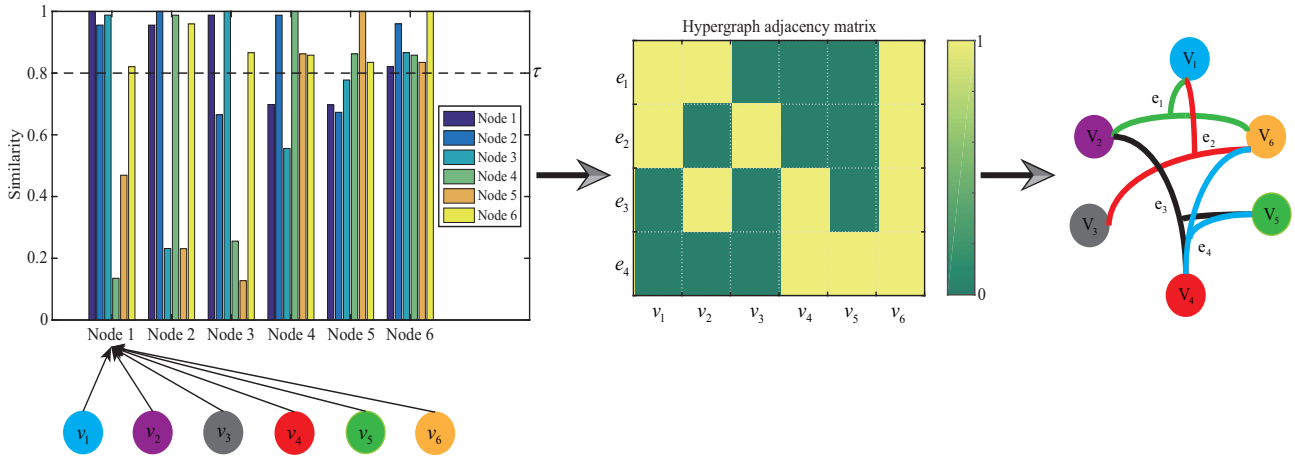

Figure 2: Hypergraph Construction and Adjacency Matrix Illustration

3.4. Hypergraph Capsule Routing Mechanism

In financial time series, market behavior often presents as a group synergy effect: multiple time points may be simultaneously influenced by the same macroeconomic event, or a group of related time points may exhibit a coordinated movement pattern that goes beyond pairwise relationships. To capture this higher-order dependency that goes beyond pairwise relationships, this paper innovatively deeply integrate the hypergraph structure into the capsule routing mechanism, as shown in figure 2.

To construct the hypergraph, first consider each time step t as a node of the hypergraph, and calculate the feature vector for each node, which is the average of all the primary capsules at that time step:

$$v_t = \frac{1}{C}\sum_{c=1}^{C} v_{t,c}^{(0)} \tag{5}$$

Next, based on the parameters generated by the meta-adapter, the comprehensive similarity between any two time points t and t' is calculated. This similarity takes into account both the pattern similarity and the temporal proximity:

$$\mathbf{sim}(t,t') = \lambda_f \cdot \frac{v_t \cdot v_{t'}}{\|v_t\|\|v_{t'}\|} + \lambda_t \cdot exp(-\frac{|t-t'|}{\tau_t}) \tag{6}$$

where the first item is the cosine similarity, which measures the similarity of feature patterns; the second item is the exponentially decaying time proximity term, suggesting that the nodes that are closer in time may have stronger correlations. λ_f and λ_t control the relative importance of the two.

Based on the calculated similarity matrix, we perform an adaptive hyper-edge generation process: for each time point t, find all other nodes t' whose similarity exceeds the threshold τ, and group these nodes into the same hyper-edge, ensuring that each hyper-edge contains at least 2 nodes. Repeat this process iteratively until approximately K hyper-edges are generated or all nodes are covered. The generated hypergraph is represented by an association matrix $H \in \{0,1\}^{T\times K}$, where $H_{t,e} = 1$ if and only if time step t belongs to hyperedge e, otherwise it is 0. This hypergraph structure explicitly encodes the complex high-order correlation relationships between time points.

The traditional capsule dynamic routing combines low-level features to form high-level concepts through iterative "protocols", but does not consider the high-order group relationships between feature points. To address this, the hypergraph structure constructed in the previous step is deeply embedded into each step of the routing mechanism calculation. There are $N = T \times C$ primary capsules $v_i^{(0)}$, and the goal is to generate M capsules $v_j^{(R)}$ with a dimension of d_h through R rounds of iterative routing. Each iteration consists of the following five closely-linked steps:

Step 1: Based on the current routing log probability $b_{ij}^{(r)}$, the sparse coupling coefficient $c_{ij}(r)$ is calculated, which represents the contribution weight of the lower-level capsule i to the higher-level capsule j:

$$c_{ij}^{(r)} = \mathbf{Sparsemax}(\mathbf{b_{ij}^{(r)}}) \tag{7}$$

where the Sparsemax function is used instead of the traditional Softmax. Its output is sparse, meaning it forces each lower-level capsule to establish strong connections only with a few of the most relevant higher-level capsules. This enhances the model's discrimination ability and computational efficiency.

Step2: In traditional routing, the prediction vector of the lower-level capsule i for the higher-level capsule j is merely $\hat{v}_{j|i} = W_{ij}v_i^{(0)}$. To achieve this, a crucial enhancement is made: when the capsule makes a prediction, it integrates the information from other capsules within its own hyperedge.

$$\hat{v}_{j|i} = W_{ij}v_i^{(0)} + \sum_{H_{t_i,e}=1} \alpha_{i,e}\cdot W_e \cdot \mathbf{Mean}(\{v_k^{(0)} | H_{t_k,e} = 1\}) \tag{8}$$

The attention weight $\alpha_{i,e}$ is obtained by taking the dot product of the capsule query vector q_i and the hyper-edge

key vector k_e, and then normalizing it through Softmax:

$$\alpha_{i,e} = \frac{exp(q_i^{\mathrm{T}} k_e)}{\sum_{H_{t_i,e'}=1} exp(q_i^{\mathrm{T}} k_{e'})} \tag{9}$$

Step 3: Sum up all the enhanced prediction vectors of the low-level capsules by weighting them according to their coupling coefficients, and obtain the original input of the high-level capsules:

$$s_j^{(r)} = \sum_{i=1}^{N} c_{ij}^{(r)} \hat{v}_{j|i}^{(r)} \tag{10}$$

Then, the same squashing function is applied for non-linear activation, resulting in the output of the advanced capsules for this round of iteration:

$$v_j^{(r)} = \mathbf{Squash}(s_j^{(r)}) \tag{11}$$

Step 4: In order to establish a bidirectional optimization loop between "capsule representation" and "hypergraph structure", this paper utilizes the newly generated high-level capsules to reconstruct the feature representation of the associated hyperedges. For each hyperedge e, we aggregate all the high-level capsules that are related to it:

$$e_e^{(r)} = \mathrm{Mean}(v_j^{(r)} \mid \exists i \text{ s.t. } H_{t_i,e} = 1 \text{ and } c_{ij}^{(r)} > 0) \tag{12}$$

Then, calculate the difference between this round of reconfiguration and the previous round as the reconfiguration loss:

$$L_{rec,e}^{(r)} = \|e_e^{(r)} - e_e^{(r-1)}\|_2^2 \tag{13}$$

where the loss measures the stability of the super-edge representation, where $e_e^{(0)}$ is obtained through the polymerization of the primary capsules.

Step 5: Finally, based on the consistency principle and the super-edge feedback, the routing log probability b_{ij} is updated. The update rule not only considers the dot product of the predicted vector $\hat{v}_{j|i}^{(r)}$ and the current high-level capsule output $v_j^{(r)}$, but also introduces the super-edge reconstruction loss as a penalty term:

$$b_{ij}^{(r+1)} = b_{ij}^{(r)} + \hat{v}_{j|i}^{(r)} \cdot v_j^{(r)} - \gamma \cdot \sum_{H_{t_i,e}=1} L_{rec,e}^{(r)} \tag{14}$$

where $\cdot$ represents the dot product operation, and γ is a learnable equilibrium parameter. This mechanism is of crucial importance: it encourages connections that not only enable local consistency among capsules but also contribute to the formation of stable and consistent group representations at the super-level.

After such an iterative process of R rounds, a stable set of advanced capsules with complex high-order dependency relationships is ultimately obtained:

$$V^{(R)} = \{v_1^{(R)}, v_2^{(R)}, \cdots, v_M^{(R)}\} \tag{15}$$

3.5. Read out and predict

After obtaining the high-level capsule collections, a decoding mechanism is needed to convert them into global representations for prediction. We adopt adaptive attention pooling. The model learns a global query vector q_r, by calculating the dot product between it and each high-level capsule and normalizing it, to obtain the attention weights of each capsule:

$$w_j = \frac{exp(q_r^{\mathrm{T}} \cdot v_j^{(R)})}{\sum_{j'=1}^{M} exp(q_r^{\mathrm{T}} \cdot v_{j'}^{(R)})} \tag{16}$$

This essentially enables the model to determine by itself which high-level patterns are more important in the current prediction context. After weighted summation, a global context vector is obtained, and at the same time, the prediction values for the next P time steps are directly output through a MLP prediction network:

$$z = \mathbf{MLP}(\sum_{j=1}^{M} w_j \cdot v_j^{(R)}) \tag{17}$$

3.6. Meta-training strategy

In order to effectively train the meta-adaptor and enable it to generate highly generalized dynamic parameters based on sequence features, this paper adopts the model-agnostic meta-learning (MAML) framework and designs a curriculum learning strategy[34, 35]. Firstly, a large number of meta-learning tasks $\{\mathcal{T}_i\}$ are constructed from historical data. Each task corresponds to a segment of historical data of a stock and is divided into a support set D_i^{supp} and a query set D_i^{query}. The training is divided into two stages: the inner-loop adaptation and the outer-loop meta-optimization.

Inner-loop adaptation: For task $\mathcal{T}_i$, forward propagation is conducted using the support set data, and the meta-adaptor generates parameters Θ_i. The model uses Θ_i to calculate the loss $\mathcal{L}_{supp}$ on the support set and performs only one step of gradient descent on the main network parameters θ to obtain the adapted parameters θ_i'. During this process, the parameters ϕ of the meta-adaptor are frozen.

Outer-loop meta-optimization: Using the adapted main network parameters θ_i' and the same meta-parameters Θ_i, the loss $\mathcal{L}_{query}$ is calculated on the query set. This loss reflects the quality of Θ_i and provides the gradient for the update of the meta-adaptor parameters ϕ. By aggregating the gradients of multiple tasks and simultaneously updating the main network parameters θ and the meta-adaptor parameters ϕ, the meta-adaptor can learn to generate parameters that enable the model to quickly adapt to new tasks.

To describe the meta-learning training process more intuitively, this paper formalizes the meta-learning training process in the form of algorithm pseudocode, and provides clear and reproducible details for the algorithm implementation. The specific details can be found in Algorithm 1.

Algorithm 1 Meta-Learning Training for Meta-HCapsNet

Require:

Training dataset: $\mathcal{D} = \{historical\ stock\ data\}$.
Meta-adapter parameters: ϕ.
Main network parameters: θ.
Meta-learning rate: β.
Inner-loop learning rate: α.
Number of curriculum stages: $S = 3$.
Batch size for task sampling: B.
Number of inner-loop adaptation steps: K *(usually* $K = 1$*).*

Ensure:

Trained parameters: ϕ, θ.
Initialize
Randomly initialize ϕ and θ
Set curriculum stage: $s \leftarrow 1$ (stable) $\rightarrow s \leftarrow 2$ (transition) $\rightarrow s \leftarrow 3$ (volatile)
for epoch = 1 **to** MaxEpochs **do**
  tasks $\leftarrow$ SampleTasks($\mathcal{D}$, B, stage = s)
  Initialize meta-gradients: $\nabla_{\phi}^{\text{meta}} \leftarrow \mathbf{0}$, $\nabla_{\theta}^{\text{meta}} \leftarrow \mathbf{0}$
  total_loss $\leftarrow 0$
  for each task $\mathcal{T}_i$ in tasks **do**
    $\theta_i' \leftarrow \theta$ {Copy main network parameters}
    for step = 1 **to** K **do**
      $(X_{\text{supp}}, y_{\text{supp}}) \leftarrow \mathcal{T}_i$.support_set
      $\hat{y}_{\text{supp}} \leftarrow$ Meta-HCapsNet$(X_{\text{supp}}; \phi, \theta_i')$
      $\mathcal{L}_{\text{supp}} \leftarrow$ ComputeLoss$(y_{\text{supp}}, \hat{y}_{\text{supp}})$
      $\nabla_{\theta_i'} \leftarrow \partial \mathcal{L}_{\text{supp}} / \partial \theta_i'$
      $\theta_i' \leftarrow \theta_i' - \alpha \cdot \nabla_{\theta_i'}$
    end for
    $(X_{\text{query}}, y_{\text{query}}) \leftarrow \mathcal{T}_i$.query_set
    $\hat{y}_{\text{query}} \leftarrow$ Meta-HCapsNet$(X_{\text{query}}; \phi, \theta_i')$
    $\mathcal{L}_{\text{query}} \leftarrow$ ComputeLoss$(y_{\text{query}}, \hat{y}_{\text{query}})$
    $\nabla_{\phi_i} \leftarrow \partial \mathcal{L}_{\text{query}} / \partial \phi$
    $\nabla_{\theta_i} \leftarrow \partial \mathcal{L}_{\text{query}} / \partial \theta$
    $\nabla_{\phi}^{\text{meta}} \leftarrow \nabla_{\phi}^{\text{meta}} + \nabla_{\phi_i}$
    $\nabla_{\theta}^{\text{meta}} \leftarrow \nabla_{\theta}^{\text{meta}} + \nabla_{\theta_i}$
    total_loss $\leftarrow$ total_loss $+ \mathcal{L}_{\text{query}}$
  end for
  $\nabla_{\phi}^{\text{meta}} \leftarrow \nabla_{\phi}^{\text{meta}} / B$
  $\nabla_{\theta}^{\text{meta}} \leftarrow \nabla_{\theta}^{\text{meta}} / B$
  $\phi \leftarrow \phi - \beta \cdot \nabla_{\phi}^{\text{meta}}$
  $\theta \leftarrow \theta - \beta \cdot \nabla_{\theta}^{\text{meta}}$
  LogLoss(epoch, total_loss$/B$)
  if epoch % StageSwitchInterval = 0 **then**
    $s \leftarrow s + 1$
    if $s > 3$ **then**
      $s \leftarrow 3$
    end if
  end if
end for
return ϕ, θ

4. Experimental analysis

To verify the generalization capability and robustness of the proposed framework across diverse market sectors, this study employs empirical data from three distinct, high-liquidity entities listed on the Chinese stock market. These targets are carefully selected to represent the Automotive, Electrical Equipment, and Shipbuilding industries, respectively, ensuring a broad evaluation of the model's adaptability to different industrial cycles .

Specifically, the Opening Price is utilized as the primary modeling variable, as it encapsulates critical information regarding overnight market sentiment and investor expectations. The detailed statistics of the selected datasets, including the covered time periods and the total number of trading days, are summarized in Table 1.

Given its pivotal role in aggregating market information, the opening price is adopted as the key input feature. The fluctuation patterns of the empirical data are depicted in Figure 3.

4.1. Comparative Analysis

To rigorously evaluate the forecasting proficiency and robustness of the proposed Meta-HCapsNet, we executed a comprehensive comparative analysis against seven distinct baseline models across three real-world stock datasets: BYD, CATL, and CSSC. The selection of baselines encompasses the evolutionary trajectory of financial time series modeling, ensuring a holistic performance assessment:

Traditional Machine Learning Methods: We selected SVM and RF to establish a baseline for non-linear regression capabilities without deep feature extraction.

Deep Sequence and Hybrid Models: We included the standard LSTM as a fundamental recurrent baseline. Furthermore, to represent advanced hybrid architectures, we employed BiCuDNNLSTM-1dCNN , which integrates bidirectional memory with convolutional feature extraction, and the CNN-LSTM-Attention model, which utilizes multi-head attention to weight temporal features dynamically.

SOTA Transformer Architectures: We compared against iTransformer , which inverts dimensions to capture multivariate correlations, and MEAformer, a cutting-edge model that utilizes external attention mechanisms to reduce complexity and enhance long-term forecasting.

To ensure reproducibility and fair comparison, all models were implemented using PyTorch on an NVIDIA GPU platform. The specific hyperparameter configurations for each model, determined through grid search and literature recommendations, are detailed in Table 2.

The prediction accuracy was evaluated using three standard metrics: Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) to measure the magnitude of errors, and Mean Absolute Percentage Error (MAPE) to assess the relative deviation, as shown in equation 18. Lower values in these metrics indicate superior model performance. The experiments were conducted under a unified sliding-window

Table 1
Statistics of Experimental Datasets

Company	Industry Sector	Time Period	Samples
BYD Automobile Co., Ltd. (BYD)	the automotive vehicle industry	2011/06/30-2025/01/10	3278
Contemporary Amperex Technology Co. Ltd.(CATL)	the electrical equipment industry	2018/06/11-2025/01/10	1601
Contemporary Amperex Technology Co. Ltd.(CATL)	the shipbuilding industry	1998/05/20-2025/01/10	5854

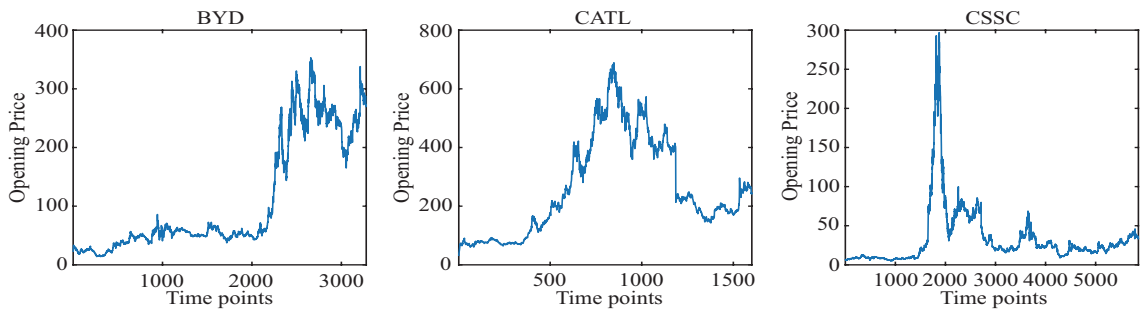

Figure 3: Opening Price Time Series of Three Target Stocks

framework to ensure fair comparison.

$$\begin{cases} \mathbf{MSE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \\ \mathbf{MAPE} = \frac{100\%}{N}\sum_{i=1}^{N}|\frac{y_i-\hat{y}_i}{y_i}| \\ \mathbf{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} \end{cases} \tag{18}$$

where y_i and $\hat{y}_i$ represent the true value and the predicted value respectively.

The quantitative results are presented in Table 3. As demonstrated in Table 3, Meta-HCapsNet consistently outperforms all baselines across the three datasets. The results

Table 2
Hyperparameter Settings of Compared Models

Category	Model	Hyperparameter	Value / Setting
Common Setup	all models	input window(T)/horizon(p)	30 / 5
		Optimizer / Learning Rate	Adam / $1e^{-3}$ (decay 0.5 per 10 epochs)
		Batch Size / Epochs	32 / 100 (Early Stopping: 10)
Traditional ML	SVM	Kernel / C / Epsilon	RBF / 100 / 0.1
	RF	estimators / Max Depth	200 /Full tree
Deep / Hybrid	LSTM	Hidden Dim / Layers	64 / 2
	BiCuDNNLSTM-1dCNN	Bi-LSTM Units	128
		CNN Filters / Kernel Size	64 / 3
	CNN-LSTM-Attention	CNN Filters	64
		LSTM Units	128
		Attention Heads	4
Transformers	iTransformer	Model Dim	512
		Encoder Layers	2
	MEAformer	Memory Size	64
		External Attention Heads	8
Proposed	Meta-HCapsNet	Primary / High-level Capsule Dim	$d_p = 8$ / $d_h = 16$
		Routing Iterations	3
		Meta-Learning Stages	3
		Hypergraph Edges	Adaptive

Table 3
Performance Comparison of Meta-HCapsNet with Baseline Models on Three Datasets

Model	BYD			CATL			CSSC		
	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
SVM	16.26	20.96	5.79%	15.95	21.13	6.88%	5.36	7.05	11.19%
RF	15.14	19.45	5.32%	15.10	20.04	6.52%	5.01	6.88	10.45%
LSTM	12.13	15.87	4.36%	13.09	17.58	6.07%	3.36	4.23	6.87%
BiCuDNNLSTM-1dCNN	10.85	14.12	3.92%	12.88	17.95	5.65%	2.85	3.56	5.72%
CNN-LSTM-Attention	10.42	13.75	3.75%	12.94	17.65	5.58%	2.56	3.28	5.14%
iTransformer	9.57	12.74	3.36%	13.01	17.81	5.59%	1.93	2.53	4.14%
MEAformer	9.25	12.10	3.28%	12.85	17.40	5.48%	2.05	2.68	3.95%
Meta-HCapsNet	8.84	11.64	3.17%	12.57	17.01	5.43%	1.17	1.46	2.46%

presented in Table **??** confirm that Meta-HCapsNet consistently secures the leading performance across all three datasets.

In comparison with hybrid deep learning models like BiCuDNNLSTM-1dCNN and CNN-LSTM-Attention, our proposed architecture delivers a substantial reduction in prediction error. Specifically on the CSSC dataset, Meta-HCapsNet lowers the MAE from 2.56 to 1.17 relative to the CNN-LSTM-Attention baseline, representing an improvement of approximately 54%. This evidence suggests that while hybrid models are capable of local feature extraction and temporal memory, their reliance on static geometric structures limits their ability to capture dynamic "group synergy" effects. Conversely, the Hypergraph Capsule Routing mechanism in our model explicitly encodes these high-order dependencies to better identify subtle market fluctuations.

Furthermore, the proposed model outperforms state-of-the-art Transformer variants like iTransformer and MEAformer. While MEAformer shows competitive accuracy on the BYD dataset by achieving an MAE of 9.25, its rigid structure limits adaptability to varying market conditions. Our Meta-Adapter addresses this by dynamically generating routing priors based on statistical features, allowing for tailored processing of high-volatility assets like CATL and stable assets like CSSC. This capability is evidenced by the superior RMSE of 17.01 achieved on the volatile CATL dataset, which validates the enhanced robustness and generalization of our approach compared to conventional attention-based architectures.

4.2. Robustness Analysis across Extended Forecasting Horizons

To rigorously evaluate the long-term predictive stability and performance attenuation of Meta-HCapsNet, we expanded the forecasting horizon P from the initial 5 days to a range of $\{5, 10, 20, 30, 40\}$ trading days. The historical look-back window T was maintained at 30 days to examine how the model handles increasing uncertainty with limited historical input. We compared Meta-HCapsNet against two state-of-the-art baselines, iTransformer and MEAformer, across three datasets.

Figure 4 shows the changes in MAE for these three models as the prediction period lengthens. Firstly, it can be observed that the upward trend of Meta-HCapsNet is the most gradual. Taking BYD as an example, the MAE of iTransformer increased by approximately 123.2% from $P = 5$ to $P = 40$, while Meta-HCapsNet only increased by 107.5%. This directly proves that the high-order group correlation captured by the hypergraph capsule routing is more capable of resisting the non-stationarity and noise interference of financial data compared to the traditional point-to-point Attention.

Secondly, it can be observed that on the highly fluctuating CATL dataset, the differences among the various models become increasingly obvious as P increases. The dynamically generated routing prior Θ_r by Meta-Adapter plays a crucial role here. It can adjust the parameters in real time based on the statistical characteristics of the input sequence, enabling the model to still accurately adapt to the drift of market styles in long-term predictions. On the CSSC dataset, when $P = 40$, Meta-HCapsNet still keeps the MAE at a low level, while the comparison models have all exceeded 6.0. This stable performance indicates that the capsule network's encoding ability for entity posture information is more resistant to attenuation compared to simple scalar feature fusion in multi-step predictions.

4.3. Ablation experiment

To thoroughly validate the effectiveness of each key component in Meta-HCapsNet, we conducted comprehensive ablation experiments on the BYD, CATL, and CSSC datasets. All ablation variants share the same overall experimental settings as the full model, including a history window length$T = 30$, prediction horizon$P = 5$. We constructed six ablation variant models. Each variant removes or replaces only one key component while keeping all others unchanged to isolate the contribution of the targeted component:

without Meta-Adapter: This variant completely removes the Meta-Adapter module. Hypergraph convolution parameters, including weight matrices, and capsule routing priors, including initial routing coefficients, are set to fixed values that are randomly initialized and frozen throughout training instead of being dynamically generated from the input. The

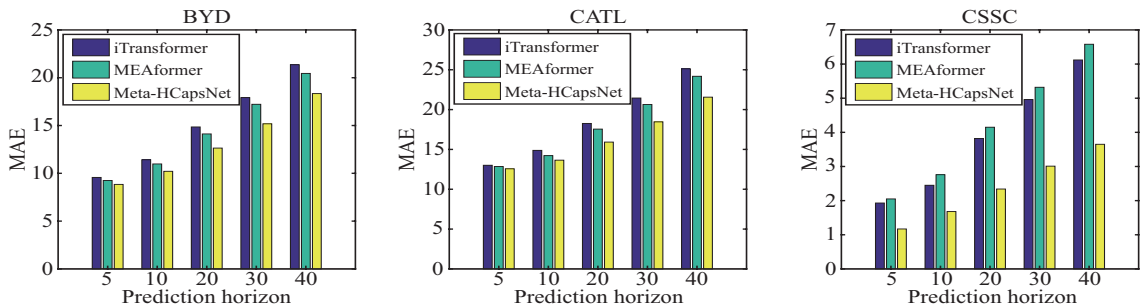

Figure 4: MAE Comparison Across Different Prediction Horizons

model therefore loses its ability to rapidly adapt to different stock or market styles.

without Hypergraph: This variant replaces the hypergraph structure with a standard fully connected pairwise graph. Hyperedges are decomposed into all possible pairwise edges, and convolution operations degrade to standard graph convolutional networks, retaining only second-order node relationships and losing the ability to model higher-order group synergies.

without Capsule Routing: This variant removes the dynamic capsule routing mechanism. Feature aggregation in the capsule layer is replaced with a standard multilayer perceptron consisting of two hidden layers with ReLU activation that performs linear transformation and summation directly on input capsules, eliminating the iterative agreement-based routing and encoding of entity existence probabilities.

without Attention Pooling: This variant removes the adaptive attention pooling module. The final global representation is obtained by simple average pooling over all high-level capsules followed by a single-layer multilayer perceptron with hidden dimension matching the original attention head output, losing the ability to dynamically weight important capsules according to market context.

without High-order Aggregation: During hypergraph capsule routing, this variant uses only the features of the central or representative node within each hyperedge for updates, ignoring contributions from other associated nodes in the hyperedge. This effectively reduces processing to near-independent node handling and weakens higher-order group information fusion.

without Reconstruction Loss: This variant removes the hyperedge reconstruction loss term during routing updates, retaining only the main prediction loss for backpropagation. The routing iteration lacks bidirectional constraints and becomes prone to instability in group representations.

All variables were evaluated using the same 30-day input window and 5-day prediction range. The results are shown in Table 4.

The ablation results reveal the critical contributions of each component. Removing Meta-Adapter causes the most severe degradation, with relative MAE increase of 13.7% and RMSE increase of 13.6% on the CATL dataset, demonstrating that dynamic parameter generation by Meta-Adapter is critical for handling market non-stationarity and stock heterogeneity, as fixed parameters fail to adapt effectively to different volatility patterns. Replacing hypergraph with pairwise graph leads to moderate-to-large degradation, with average MAE increase of 6-10%, and the effect is particularly pronounced on BYD and CATL, validating the unique advantage of hypergraphs over fully connected graphs in capturing multi-node higher-order relationships such as industry chain or sector collective movements. Removing capsule routing produces one of the largest drops, with MAE increase of 38.5% on CSSC, highlighting the irreplaceable role of dynamic routing iterations in encoding high-level semantic capsules with entity pose and existence probability, while standard multilayer perceptron aggregation loses the interpretable feature composition process. Removing attention pooling results in stable but noticeable degradation, with average MAE increase of 4-6%, as adaptive attention effectively highlights key group capsules under current market conditions, whereas average pooling produces overly smoothed global representations and reduces prediction precision. Removing high-order aggregation shows significant degradation on high-volatility datasets, with MAE increase of 11.5% on CATL, confirming that integrating multi-node information within hyperedges is essential for modeling collective movement patterns, and relying solely on individual node features severely weakens group synergy capture. Removing reconstruction loss produces the smallest yet consistent degradation, with average MAE increase of 3-5% across all datasets, as the auxiliary constraint substantially improves routing stability and convergence quality, preventing local oscillations that arise from unconstrained iteration.

4.4. Few-Shot Adaptation Experiment

To further validate the meta-learning capability of the Meta-Adapter in enabling rapid adaptation to new market regimes with limited historical data, we conducted few-shot adaptation experiments. These experiments simulate real-world scenarios where only scarce data is available for a new stock or during emerging market shifts. We adopted a leave-one-out protocol. The model was first pre-trained on two datasets, BYD and CSSC, using the full meta-learning procedure for Meta-HCapsNet and standard end-to-end training for the fixed-adapter baseline. The target dataset, CATL, was

Table 4
Ablation Study Results

Model	BYD			CATL			CSSC		
	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
Meta-HCapsNet	8.84	11.64	3.17%	12.57	17.01	5.43%	1.17	1.46	2.46%
w Meta-Adapter	9.92	13.05	3.58%	14.28	19.32	6.15%	1.48	1.85	3.12%
w Hypergraph	9.45	12.48%	3.41%	13.86	18.74	5.92%	1.36	1.69	2.89%
w Capsule Routing	10.38	13.72	3.79%	14.95	20.16	6.48%	1.62	2.03	3.45%
w Attention Pooling	9.26	12.21	3.32%	13.14	17.82	5.68%	1.29	1.61	2.73%
w High-order Aggregationt	9.68	12.79	3.49%	14.02	18.95	6.03%	1.41	1.76	2.97%
w Reconstruction Loss	9.12	12.03	3.26%	12.89	17.45	5.56%	1.24	1.55	2.62%

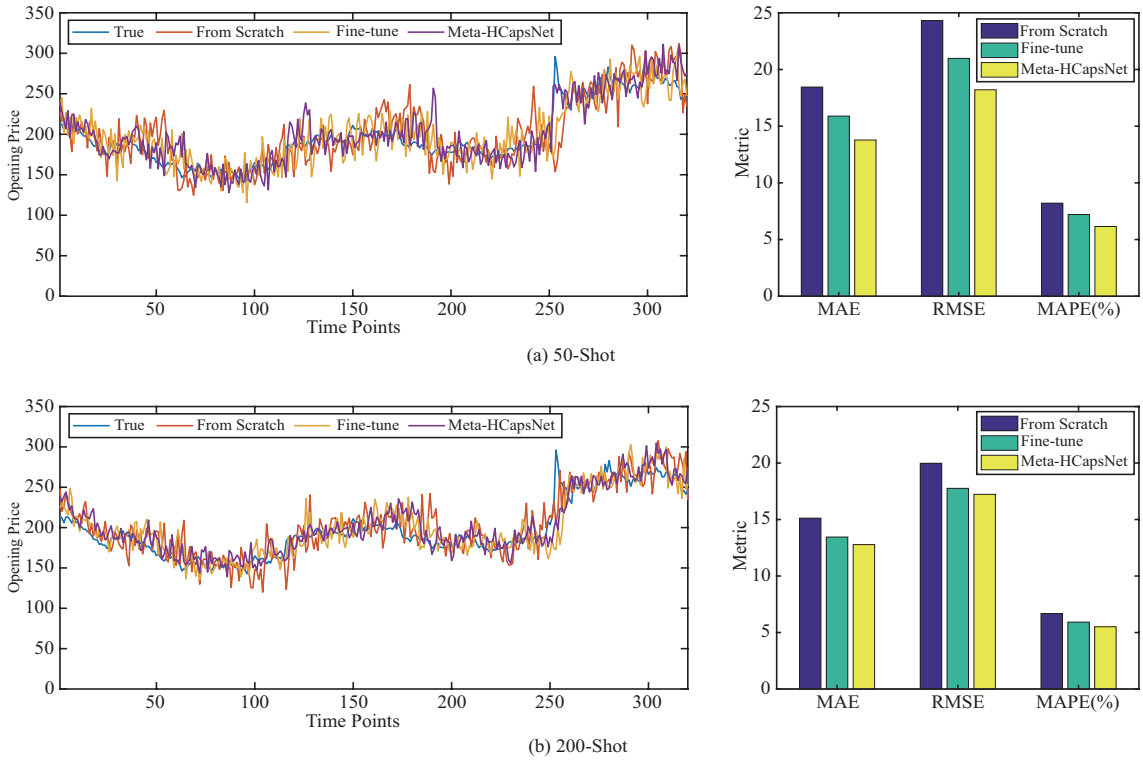

Figure 5: Few-Shot Adaptation Results Comparison

then used for adaptation with varying amounts of support data—specifically 50-shot, and 200-shot, corresponding to 50 and 200 trading days of training sequences. Adaptation follows the MAML inner-loop process with three gradient steps on the support set for the meta-learning version, while the baseline uses standard fine-tuning. This paper compare three settings:

From Scratch: The model is initialized randomly and trained only on the limited support set of the target stock.

Fixed-Adapter Fine-tune: The model is pre-trained without meta-learning, then fine-tuned on the support set.

Meta-HCapsNet: The full meta-trained model performs few-shot inner-loop adaptation using dynamically generated parameters from the Meta-Adapter.

Figure 5 shows the visualization of the predicted values and true values for the three settings in the 50-shot and 200-shot scenarios, as well as the presentation of three metrics. The results demonstrate that Meta-HCapsNet significantly outperforms both baselines in all few-shot regimes, with the advantage most pronounced under extremely limited data. In the 50-shot setting, Meta-HCapsNet reduces MAE by 25.3% relative to training from scratch and by 13.3% relative to fixed-adapter fine-tuning. This substantial improvement stems from the Meta-Adapter's ability to generate tailored hypergraph parameters and routing priors based on input statistical features, allowing only a few gradient steps to effectively align the model with the new market's non-stationary dynamics. The fixed-adapter variant, despite benefiting from pre-training, suffers from rigid parameters that

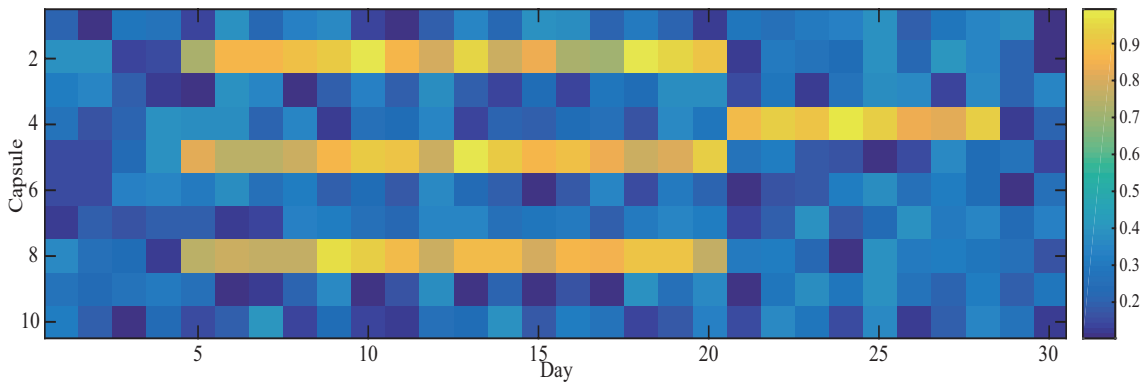

Figure 6: Activation Intensity Heatmap of High-Level Capsules in Rising-Retracement Phases

cannot quickly accommodate the high volatility and distinct patterns of CATL. Training from scratch performs worst, as expected, due to severe overfitting on scarce data and lack of transferable knowledge.

As the number of shots increases, the performance gap narrows but remains consistent, with Meta-HCapsNet achieving near full-data performance already at 200 shots. These findings robustly validate the effectiveness of the MAML-based meta-training strategy: the Meta-Adapter learns to produce adaptive parameters that enable fast generalization, making the model particularly suitable for practical financial applications where data for new assets or regime shifts is initially limited.

4.5. Interpretability Analysis

Although quantitative metrics demonstrate the superior predictive performance of Meta-HCapsNet, its capsule-based architecture and hypergraph integration also provide rich interpretability, addressing a critical limitation of existing black-box deep learning models in financial applications. In this section, we qualitatively analyze the internal representations of the model to illustrate how high-level capsules and hypergraph structures offer intuitive explanations of market states and decision-making processes.

The magnitude of each high-level capsule vector explicitly represents the existence probability of a specific high-order market pattern. We visualize the activation strengths of all high-level capsules over a representative 30-day input window on the BYD dataset, during which the stock experienced a clear upward trend followed by a short correction

As shown in Figure 6, capsules 2, 5, and 8 exhibit consistently high activation during the rising phase, suggesting they encode bullish patterns such as sustained buying pressure or positive sector momentum. In contrast, capsule 4 activates strongly during the correction phase, potentially corresponding to risk-averse or profit-taking behaviors. This pattern is consistent across multiple windows: capsules with high activation during upward movements tend to have pose vectors that align with increasing trend intensity statistics extracted by the Meta-Adapter.

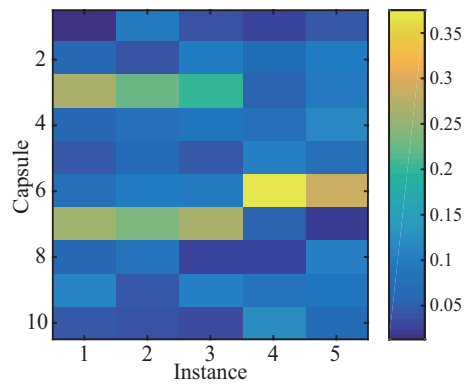

Figure 7: Weight Distribution of High-Level Capsules under Adaptive Attention Pooling

The adaptive attention pooling mechanism further enhances interpretability by revealing which high-level patterns the model considers most relevant for the final prediction. Figure 7 displays the attention weights assigned to each capsule for five consecutive prediction instances on the CATL dataset during a volatile period. We observe that in instances where the price rises sharply, the model assigns significantly higher weights to capsules 3 and 7, which were previously identified as bullish capsules. Conversely, during downward movements, weight shifts toward capsule 6. This dynamic weighting provides stakeholders with clear explanations: the model's prediction is primarily driven by specific interpretable high-order patterns rather than opaque aggregated features.

5. Conclusion

In this study, we proposed Meta-HCapsNet, a novel attention-enhanced capsule network with hypergraph structure to tackle the challenges of stock price sequence forecasting, including high noise, non-stationarity, and complex high-order interdependencies in financial time series. The model integrates three key innovations: a lightweight Meta-Adapter for rapid meta-learning adaptation, a Hypergraph

Capsule Routing layer for modeling group-level correlations, and an adaptive attention pooling mechanism for improved interpretability. Experiments on three real-world Chinese stock datasets (BYD, CATL, and CSSC) show that Meta-HCapsNet consistently outperforms seven state-of-the-art baselines, including iTransformer and MEAformer, achieving up to 54% reduction in MAE and substantial improvements in RMSE and MAPE across various horizons. Ablation studies confirm the effectiveness of each component, few-shot experiments demonstrate strong generalization with limited data, and interpretability analysis reveals capsule activations aligning with meaningful market phases such as rising and retracement.

These results highlight Meta-HCapsNet's superior accuracy, adaptability, and transparency in modeling complex financial dynamics. Future work will explore extensions to multivariate forecasting, integration of multimodal data such as news sentiment, and applications to other non-stationary domains.

CRediT authorship contribution statement

Yucheng Wang[†]**:** Investigation, Writing-review & revising. **Zhunyan He**[†]**:** Literature summary, methodology, writing-original draft. **Kun Yu:** Writing-review & editing, supervision.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by

References

[1] X. Kong, Z. Chen, W. Liu, and etal. Deep learning for time series forecasting: A survey. *International Journal of Machine Learning and Cybernetics*, pages 1–60, 2025.

[2] R. P. Masini, M. C. Medeiros, and E. F. Mendes. Machine learning advances for time series forecasting. *Journal of Economic Surveys*, 37(1):76–111, 2023.

[3] E Abdullayev, E Akhmetshin, Z Hajiyev, T Mamadiyarov, Khorolskaya, and L. Lydia. A financial time series forecasting model using quasi-recurrent neural networks and the crown porcupine optimizer for stock market risk prediction. *Engineering, Technology and Applied Science Research*, 15(6):29035–29040, 2025.

[4] E. Callahan. Financial time series forecasting based on sliding window–variational mode decomposition and deep learning. *Preprints*, page 1–10, 2025.

[5] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. *In Proceedings of the International Conference on Learning Representations (ICLR)*, page 1–24, 2023.

[6] Y. Liu, T. Hu, and H. etal. Zhang. itransformer: Inverted transformers are effective for time series forecasting. *In Proceedings of the International Conference on Learning Representations (ICLR)*, page 1–25, 2024.

[7] Z. Ahmad, S. Bao, and M. Chen. Deeponet-inspired architecture for efficient financial time series prediction. *Mathematics*, 12(24):3950, 2024.

[8] M. Parker, M. Ghahremani, and S. Shiaeles. Stock price prediction using a stacked heterogeneous ensemble. *International Journal of Financial Studies*, 13(4):201, 2025.

[9] R. Feng, S. Jiang, X. Liang, and M. Xia. Stgat: Spatial–temporal graph attention neural network for stock prediction. *Applied Sciences*, 15(8):4315, 2025.

[10] S. Jia, H. Gao, J. Huang, and etal. Cirgnn: Leveraging cross-chart relationships with a graph neural network for stock price prediction. *Mathematics*, 13(15):2402, 2025.

[11] Y. Duan, W. Wang, and J. Li. Factorgcl: A hypergraph-based factor model with temporal residual contrastive learning for stock returns prediction. *arXiv:2502.05218v1*, pages 1–9, 2025.

[12] P. Xiao. Stock market prediction based on financial news, text data mining, and investor sentiment analysis. *International Journal of Information System Modeling and Design*, 15(1):1947–8186, 2024.

[13] S. Agrawal, N. Kumar, G Rathee, C. A. Kerrache, C. T. Calafate, and M. Bilal. Improving stock market prediction accuracy using sentiment and technical analysis. *Electronic Commerce Research*, 25:4103–4126, 2025.

[14] W. Xie and N. Metawa. Financial stock market forecasting based on intelligent support vector machine regression model. *Journal of Intelligent and Fuzzy Systems*, page 1–10, 2021.

[15] S. Divyashree, C. J. Joshua, A. Q. Md, S. Mohan, A. S. Abdullah, U. H. Mohamad, and A. Innab, N.and Ahmadian. Enabling business sustainability for stock market data using machine learning and deep learning approaches. *Annals of Operations Research*, 342(1):287–322, 2024.

[16] Y. Xu, C. Yang, S. Peng, and Y. Nojima. A hybrid two-stage financial stock forecasting algorithm based on clustering and ensemble learning. *Applied Intelligence*, 50(11):3852–3867, 2020.

[17] I. K. Nti, A. F. Adekoya, and B. A. Weyori. A comprehensive evaluation of ensemble learning for stock-market prediction. *Journal of Big Data*, 7(20):1–40, 2020.

[18] S. S. Alotaibi. Ensemble technique with optimal feature selection for saudi stock market prediction: A novel hybrid red deer-grey algorithm. *IEEE Access*, 9:64929–64944, 2021.

[19] S. P. Padalkar, S. S. Bere, and D. B. Hanchate. Prediction of stocks and stock price using artificial intelligence. *International Journal of Creative Research Thoughts(IJCRT)*, 12:130–136, 2024.

[20] W. Long, Z. Lu, and L. Cui. Deep learning-based feature engineering for stock price movement prediction. *Knowledge-Based Systems*, 164:163–173, 2019.

[21] F. D. Paiva, Nogueira Cardoso R. T., G. P. Hanaoka, and et al. Decision-making for financial trading: A fusion approach of machine learning and portfolio selection. *Expert Systems with Application*, 115:635–655, 2019.

[22] N. Saurabh. Lstm-rnn model to predict future stock prices using an efficient. *International Research Journal of Engineering and Technology (IRJET)*, 7(11):672–677, 2020.

[23] A. Q. Md, S. Kapoor, J A.V. Chris, A.K. Sivaraman, K. F. Tee, H. Sabireen, and N. Janakiraman. Novel optimization approach for stock price forecasting using multi-layered sequential lstm. *Applied Soft Computing*, 134:109830, 2023.

[24] A. Kanwal, M.F. Lau, S.P.H. Ng, C. Huang, and X. Yao. Bicudnnlstm-1dcnn—a hybrid deep learning-based predictive model for stock price prediction. *Expert Systems with Applications*, 202:117123, 2022.

[25] A. Prasad and A. Seetharaman. Importance of machine learning in making investment decision in stock market. *Vikalpa*, 46(4):209–222, 2021.

[26] L. Sun, W. Xu, J. Liu, and H. Wang. Two-channel attention mechanism fusion model of stock price prediction based on cnn-lstm. *Transactions on Asian and Low-Resource Language Information Processing*, 20(5):1–12, 2021.

[27] S. Huang, Y. Liu, H. Cui, and etal. Meaformer: An all-mlp transformer with temporal external attention for long-term time series forecasting.

Information Sciences, 669:120605, 2024.

[28] S. Kim and E. Park. Stad-gcn: Spatial–temporal attention-based dynamic graph convolutional network for retail market price prediction. *Expert Systems with Applications*, 255:124553, 2024.

[29] Z. Yang, T. Zhao, S. Wang, and etal. Mdf-dmc: A stock prediction model combining multi-view stock data features with dynamic market correlation information. *Expert Systems with Applications*, 238:122134, 2024.

[30] S. Xiang, D. Cheng, C. Shang, and etal. Temporal and heterogeneous graph neural network for financial time series prediction. *Proceedings of the 31st ACM international conference on information and knowledge management*, pages 3584–3593, 2022.

[31] T. T. Huynh, M. H. Nguyen, T. T. Nguyen, and etal. Efficient integration of multi-order dynamics and internal dynamics in stock movement prediction. *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*, pages 850–858, 2023.

[32] S. Sabour, N. Frosst, and G. E. Hinton. Dynamic routing between capsules. *Advances in neural information processing systems*, 30:3856–3866, 2017.

[33] V. Mazzia, F. Salvetti, and M. Chiaberge. Efficient-capsnet: Capsule network with self-attention routing. *Scientific reports*, 11(1):14634, 2021.

[34] R. Zhan, X. Liu, and D. F.and etal. Wong. Meta-curriculum learning for domain adaptation in neural machine translation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(16):14310–14318, 2021.

[35] Z. Chen and Y. Wang. Terminology-enriched meta-curriculum learning for domain neural machine translation. *International Conference on Intelligent Computing. Singapore: Springer Nature Singapore*, pages 379–390, 2023.